



Interfaz de usuario

Interfaz de Usuario

Contenedores o Layouts

Elementos o Widgets

Themes

- Las interfaces o layouts son ficheros de texto en **formato XML** que contienen de forma **jerárquica** los elementos (contenedores o widgets) que se quieren mostrar en pantalla

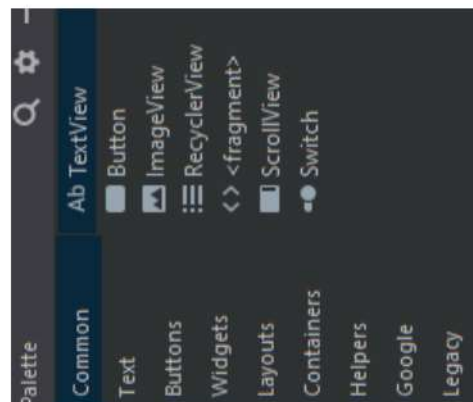
- Se almacenan en **res/layout**

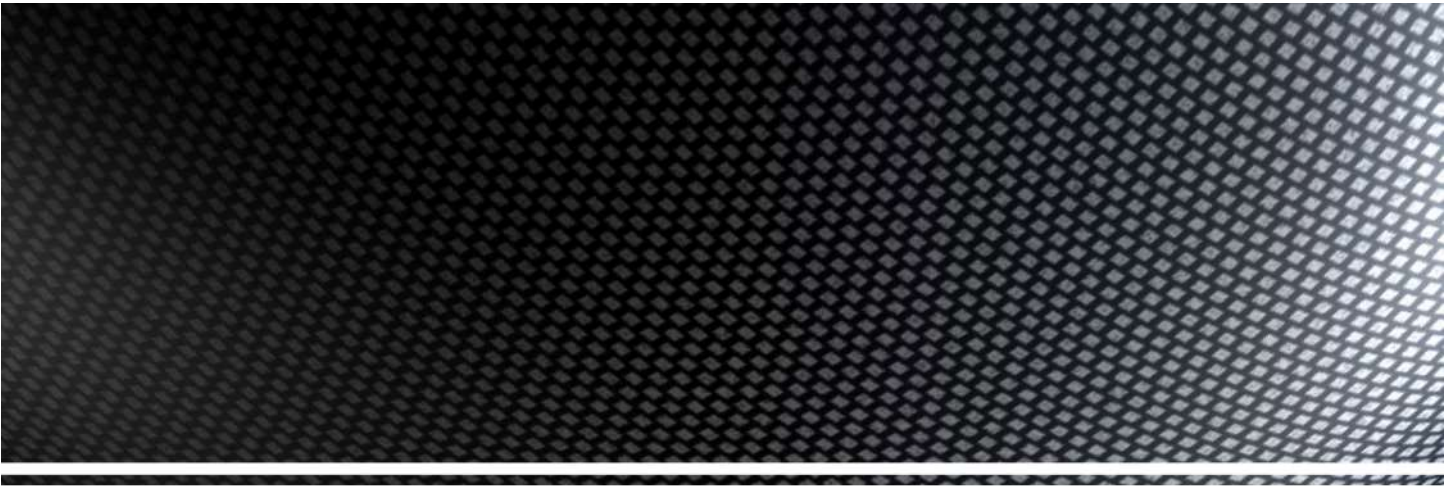


- Los **contenedores** son elementos visuales (o no) que a su vez pueden incluir otros contenedores o widgets.
 - Los **contenedores** **descienden** de la clase **ViewGroup** que a su vez descende de la clase **View**



- Los **widgets** se encuentran dentro del paquete **android.widget** y son casi todos los elementos visuales que descenden de la clase **View**





Contenedores o Layouts

Layouts

Un Layout define la estructura visual del interfaz de usuario, para **activity** y **app widgets**.

Ayudan a la separación de la representación del código.

Las definiciones son externas al código se pueden modificar sin necesidad de recompilar o modificar código.

Se pueden definir layouts para diferentes orientaciones y resoluciones.

Se pueden declarar de dos formas:

- **En XML:** En el fichero de layout con la sintaxis XML para la Clase **View** y sus subclases.
- **En runtime:** Creando objetos de tipo **View** y **ViewGroup** y manipulando sus propiedades.

Se puede acceder en runtime a los objetos y modificar sus propiedades

Layouts

La sintaxis XML para definir elementos del **UI** sigue la nomenclatura de clases y subclases, donde:

- Nombres de elementos: Corresponden con el nombre de clase
- Nombre de atributos: Corresponden con los métodos.

Ojo que hay algunas excepciones, por ejemplo:

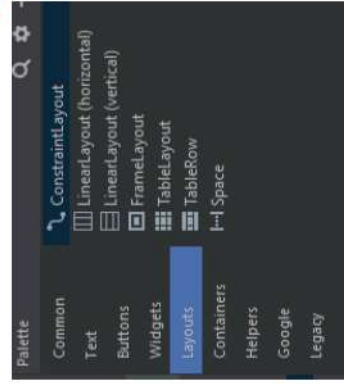
Elemento: **EditText** Atributo: **text**

Clase: **EditText** Método: **setText()**

El orden y anidación del la creación de elementos en el XML determina el orden y anidación del interfaz resultante.

Cada Layout debe contener un elemento **root** que puede ser:

- **View**
- **ViewGroup** →



```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="¡Nuevo Hola Mundo!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintHorizontal_bias="0.498"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Layouts

Ejemplo de XML:

Más información en: [Layout Resource](#)

Deben guardarse en la carpeta **res/layout** de tu proyecto con extensión .xml, por ejemplo: **main_layout.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
    <TextView android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a TextView" />
    <Button android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a Button" />
</LinearLayout>
```



El valor para el atributo ID normalmente utiliza la sintaxis: “**@+id/name**”

+: indica que es un nuevo recurso y se creará un nuevo recurso integer en

R si no existe ya. Si existe ya el recurso se omite el **+**

name: es el nombre del nuevo recurso en **R**

android: :Usando el [XML Namespace](#) (xmlns) android referenciamos a los recursos en android.R en vez de a la clase local de recursos

Layouts

El layout se compila en un **View resource** y se empaqueta en la clase R. Se puede acceder en **runtime**, al layout a partir de R por su **tipo y nombre: R.layout.<nombre>**

El layout de una Activity hay que cargarlo en tu **Activity.onCreate()** callback utilizando la función **setContentView()** pasándole el recurso layout

```
public void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.main_layout);  
}
```

Recordar que **onCreate** es un método callback que es llamado por Android cuando tu **Activity** es lanzada.

```
<TextView android:id="@+id/nameTextbox"/>
```

El recurso **nameTextbox** se compilará también en R y se puede referenciar en runtime al objeto **TextView** (del layout xml) por el nombre de su recurso en R con:

```
findViewById(R.id.nameTextbox);
```

Android Developers: [Información general sobre los recursos de las aplicaciones](#)
[Descripción general de los tipos de recursos](#)

Layouts - Recursos

Android Developers: [Información general sobre los recursos de las aplicaciones](#)
[Descripción general de los tipos de recursos](#)



Existen dos maneras de acceder a un recurso:

En código: Usando un valor entero de una subclase de tu clase R; por ejemplo:

```
R.string.hello
```

string es el tipo de recurso y hello es el nombre del recurso. Hay muchas API de Android que pueden acceder a tus recursos cuando proporcionas un ID de recurso en este formato. Consulta [Acceso a recursos en código](#).

En XML: Usando una sintaxis XML que también corresponde al ID de recurso definido en tu clase R; por ejemplo:

```
@string/hello
```

string es el tipo de recurso y hello es el nombre del recurso. Puedes usar esta sintaxis en un recurso XML en cualquier lugar donde esté previsto que proporciones un valor en un recurso. Consulta [Acceso a recursos desde XML](#).

Layouts

Ejemplo: **Definir un botón y acceder a él.**

1- Definimos la vista en el layout padre asignando un nuevo ID

```
<Button android:id="@+id/my_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/my_button_text"/>
```

2- Creamos una instancia del la vista referenciando al elemento en el **current** layout

```
Button myButton = (Button) findViewById(R.id.my_button);
```

3- Ejecución del código asociado al botón

Asociar el nombre del método asociado al evento **onClick** en las propiedades de la vista.

Definir los IDs es importante.

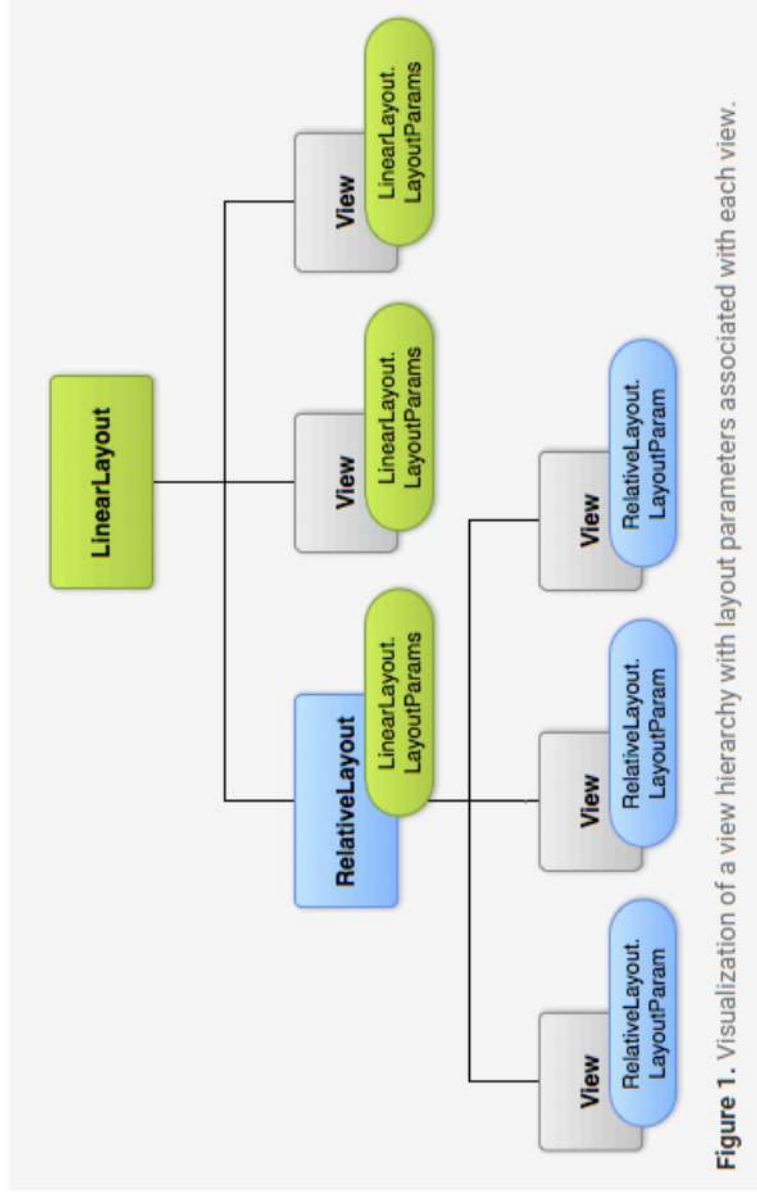
Para acceder en runtime, y porque por ejemplo, en un Relative Layout puesto que se referencian unos a otros mediante su ID

umhandroid.momrach.es: [Respondiendo al OnClick](https://umhandroid.momrach.es/Respondiendo%20al%20OnClick)

Layouts

Los **layout attributes** (**layout_something**) definen las propiedades del View o ViewGroup en el que residen.

Cada clase ViewGroup implementa una clase anidada que existe a ViewGroup.LayoutParams que contienen las propiedades **size** y **position** para cada **child view** y otras que dependen de cada caso.



Cada subclase **LayoutParams** puede definir sus propias propiedades.

También heredan **layout_width** y **layout_height** que cada **View** debe definir.

Se pueden utilizar tamaños personalizados o constantes

Layouts

Los **layout attributes** (**layout_something**) definen las propiedades del View o ViewGroup en el que residen.

Cada clase ViewGroup implementa una clase anidada que existe a ViewGroup.LayoutParams que contienen las propiedades **size** y **position** para cada **child view** y otras que dependen de cada caso.

wrap_content

Indica a la vista que se redimensione ajustándose a su contenido.

match_parent

Indica a la vista que se dimensione tan grande como permita su contenedor (el ViewGroup que lo contiene)

En general no es recomendable usar unidades absolutas como pixels.

Mejor usar unidades relativas como **dp** (**density-independent pixel units**), o **wrap_content** o **match_parent**

Android Developers:

[Cómo brindar compatibilidad con diferentes densidades de píxeles](#)

Cada subclase

LayoutParams puede definir sus propias propiedades.

También heredan

layout_width y

layout_height que cada **View** debe definir las.

Se pueden utilizar

tamaños

personalizados o constantes

Layouts

Los **layout attributes** (**layout_something**) definen las propiedades del **View** o **ViewGroup** en el que residen.

Cada clase **ViewGroup** implementa una clase anidada que existe a **ViewGroup.LayoutParams** que contienen las propiedades **size** y **position** para cada **child view** y otras que dependen en cada caso.

Una Vista es un rectángulo cuya posición se determina por el par de coordenadas **left** y **top** y dos dimensiones **width** y **height** expresados en unidades de pixel.

Se pueden obtener en runtime con **getLeft()** y **getTop()** que devuelven valores relativos a la posición del padre o contenedor.

Disponemos también de **getRight()** **getBottom()** **getWidth()** **getHeight()**

Una **View** puede definir un **padding** con **setPadding(int int int int)** (left, top, right, bottom) pero no un margin

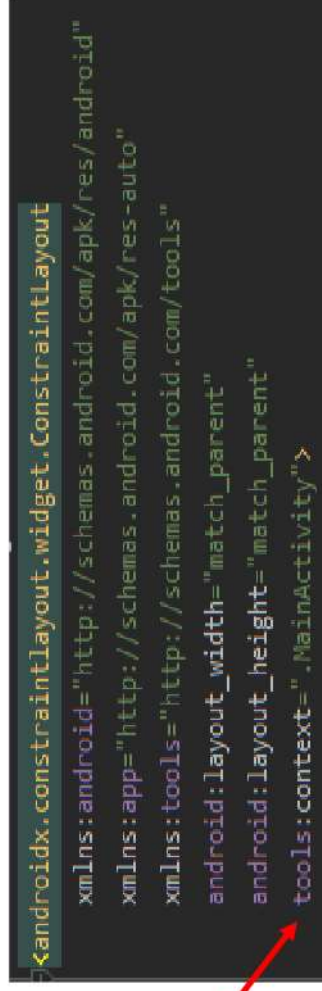
Un **ViewGroup** puede definir un **margin** ver: [ViewGroup.MarginLayoutParams](#)

Layouts - Tools

Tools : Android tiene un XML Namespace para que ciertas herramientas (como el editor visual integrado) puedan generar y guardar información en ficheros XML.

El Namespace es <http://schemas.android.com/tools> y se suele vincular con el prefijo **tools**: en los XML

El atributo **tools: context** ver más en [Tools Attributes](#)

A screenshot of an XML file in a code editor. The first line is `<androidx.constraintlayout.widget.ConstraintLayout`. Below it are several namespace declarations: `xmlns:android="http://schemas.android.com/apk/res/android"`, `xmlns:app="http://schemas.android.com/apk/res-auto"`, and `xmlns:tools="http://schemas.android.com/tools"`. Then, there are two attributes: `android:layout_width="match_parent"` and `android:layout_height="match_parent"`. The final line is `tools:context=".MainActivity">`. A red arrow points from the text "tools: context" in the paragraph above to the `tools:context` attribute in the code.

Permite :

- Que la herramienta de diseño sepa cual es la Activity de la que tiene que coger el theme (**android:Theme.Light**) que se asocia a la Activity y no al Layout.
- En el root de un XML layout se debe especificar cuál es la actividad asociada con el layout.

Se puede utilizar **.MainActivity** en vez del nombre completo del paquete de aplicación (**com.example.holaandroid.MainActivity**)

Layouts

Los **layout attributes** (**layout_something**) definen las propiedades del View o ViewGroup en el que residen.

Cada clase ViewGroup implementa una clase anidada que existe a ViewGroup.LayoutParams que contienen las propiedades **size** y **position** para cada **child view** y otras que dependen de cada caso.

Los Layouts típicos que podremos encontrar son:

- [Linear Layout](#)
- [Relative Layout](#)
- [Table Layout](#)
- [Absolute Layout](#) (deprecated)
- [Frame Layout](#)
- [List View](#)
- [Grid View](#)
- [Constraint Layout](#)

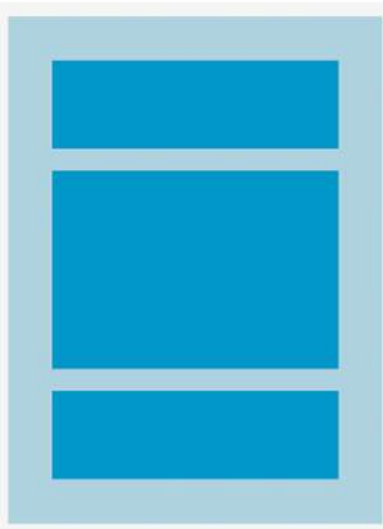


Linear Layout

ver [Linear Layout](#)

Coloca los widgets uno detrás de otro según el XML, pudiendo ser en vertical u horizontal.
Muy versátil, para interfaces sencillas.

match_parent
tras API 08



<LinearLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical">
```

</LinearLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="horizontal" >
```

<TextView

```
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:layout_weight="3"
    android:text="Nombre: " />
```

</LinearLayout>

</LinearLayout>

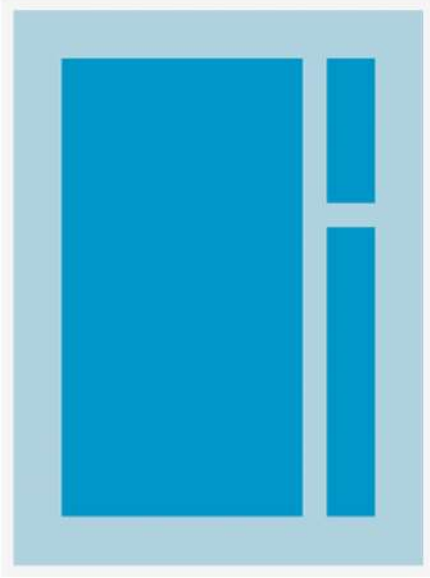
Linear Layout

Atributos

- **Background** -> permite fijar el color o imagen de fondo del contenedor.
- **Padding** -> Espacio que dejara entre el borde y los elementos de su interior.
- **Layout Margin** -> Espacio que se dejará por fuera del contenedor.
- **Layout Gravity** -> indica cómo debe actuar la gravedad (atracción) respecto al layout.

Ej. si ponemos “center” como valor gravity, se consigue un margen igual en cada uno de los bordes.

Relative Layout



ver [Relative Layout](#)

Los elementos contenidos en el se posicionan relativos unos a otros en el mismo orden en que aparecen en el fichero XML.

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >
    <TextView
        android:id="@+id/label"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_alignParentLeft="true"
        android:layout_alignParentTop="true"
        android:text="Un campo:" />
</RelativeLayout>
```


Absolute Layout

This class was deprecated in API level 3 – Use:
FrameLayout,
RelativeLayout or a
custom layout instead.

ver [Absolute Layout](#)

Podemos indicar el lugar exacto donde queramos que aparezca cada elemento.

Problemas:
cambio tamaño
pantalla,
orientación....

```
<AbsoluteLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent">
```

```
<Button
    android:layout_width="100dp"
    android:layout_height="wrap_content"
    android:text="OK"
    android:layout_x="50px"
    android:layout_y="361px" />
<Button
    android:layout_width="100dp"
    android:layout_height="wrap_content"
    android:text="Cancel"
    android:layout_x="225px"
    android:layout_y="361px" />
</AbsoluteLayout>
```

```
<AbsoluteLayout
.....
>
<TextView
    android:id="@+id/TextView01"
    android:text="Un campo:"
    android:layout_height="wrap_content"
    android:layout_width="wrap_content"
    android:layout_x="1dip"
    android:layout_y="0dip">
</TextView>
</AbsoluteLayout>
```

Relación conversión: $px = dp * (dpi / 160)$. dpi: Puntos por pulgada
dp: pixel independiente de la densidad equivalente a un pixel físico de 160 dpi.
Ej. En pantalla de 240 dpi, 1 dp equivale a 1.5 pixels físicos

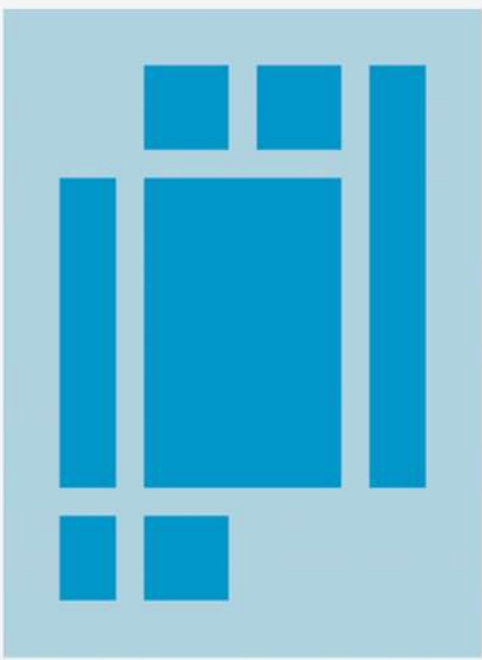
Android Developers:

[Cómo brindar compatibilidad con diferentes densidades de píxeles](#)

Table Layout

ver [Table](#) y [Table Layout](#)

Pensado para colocar los elementos en filas/columnas.
Contiene elementos tipo TableRow



<TableLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"  
android:layout_width="fill_parent"  
android:layout_height="fill_parent"  
android:shrinkColumns="1"  
android:stretchColumns="1,2" >
```

<TableRow

```
android:id="@+id/TableRow01"  
android:layout_width="match_parent"  
android:layout_height="wrap_content" >
```

<Button

```
    android:text="Button01" >
```

</Button>

<Button

```
    android:text="Button02" >
```

</Button>

</TableRow>

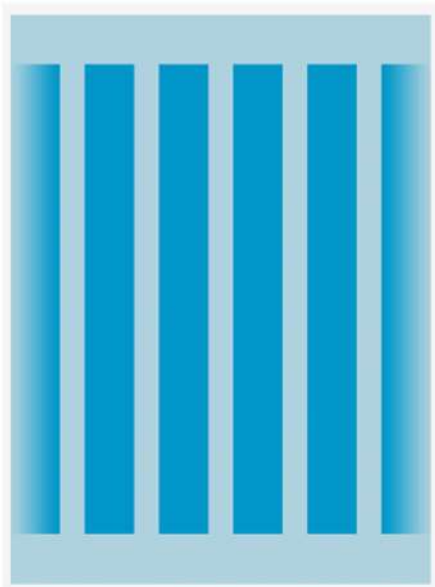
</TableLayout>

Table Layout

Atributos

- **StretchColumns**-> indica las columnas que se pueden estirar.
- **ShrinkColumns** -> indica las columnas que se pueden encoger.
- **Layout_span**-> permite que un elemento se expanda a más de una columna
- **Layout_column**-> indica en qué columna se ubicará un elemento determinado.

List View



Muestra los elementos colocados en una lista.

Los elementos se añaden a la lista utilizando un **Adapter** que inserta el contenido desde una fuente (array, base de datos, etc...) convirtiéndolo en una View dentro de la lista.

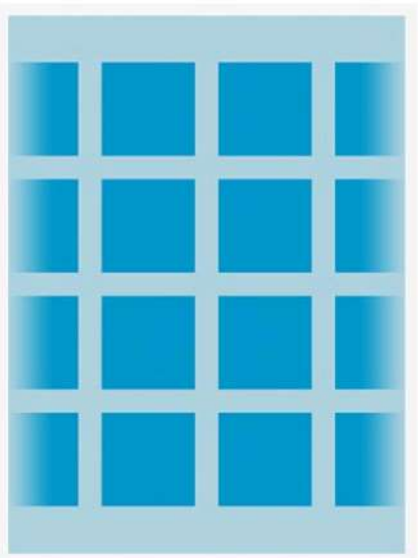
ver [List View](#)
ver [Building Layouts with an Adapter](#)

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/an
droid"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
tools:context=".ListActivity" >

<ListView
    android:id="@+id/country_list"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" >
</ListView>

</LinearLayout>
```

Grid View



ver [Grid View](#)

Muestra los elementos colocados en una matriz bidimensional.

Necesario una clase para llenar la matriz con las vistas.

Clase ListAdapter

```
<GridView
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:columnWidth="100dp"
    android:id="@+id/gridview"
    android:numColumns="auto_fit"
    android:verticalSpacing="5dp"
    android:horizontalSpacing="5dp"
    android:stretchMode="columnWidth"
    android:gravity="center"
/>
```

Frame Layout

Diseñado para enmarcar un área de la pantalla para mostrar un único elemento cuando es difícil organizar las vistas hijas para escalar a diferentes resoluciones sin superponerse.

Se pueden añadir varias vistas si controlamos su posición dentro del Frame Layout.

Lo usaremos cuando trabajemos con Fragmentes

ver [Frame Layout](#)

<FrameLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent">
```

<ImageView

```
    android:src="@drawable/ic_launcher"
    android:scaleType="fitCenter"
    android:layout_height="250px"
    android:layout_width="250px"/>
```

<TextView

```
    android:text="Frame Demo"
    android:textSize="30px"
    android:textStyle="bold"
    android:layout_height="fill_parent"
    android:layout_width="fill_parent"
    android:gravity="center"/>
```

</FrameLayout>

Tab Layout

ver [TabHost](#)

Para trabajar con pestañas.
Suele hacerse por simplicidad de código que cada pestaña sea gestionada por un **Activity** diferente.

```
<TabHost xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@android:id/tabhost"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent">
    <LinearLayout
        android:orientation="vertical"
        android:padding="10dp">
        <TabWidget
            android:id="@android:id/tabs"
            android:layout_width="fill_parent"
            android:layout_height="wrap_content" />
        <FrameLayout
            android:id="@android:id/tabcontent"
            android:layout_width="fill_parent"
            android:layout_height="fill_parent"
            android:padding="10dp" />
    </LinearLayout>
</TabHost>
```

TabHost

TabHost-> elemento principal que contendrá TabWidgets y FrameLayouts.

TabWidget -> Elemento que contiene las pestañas definidas

FrameLayout-> Contenedor usado para bloquear ciertas partes de la pantalla.
Por ejemplo, el espacio para mostrar los diferentes contenidos de cada pestaña (la seleccionada).

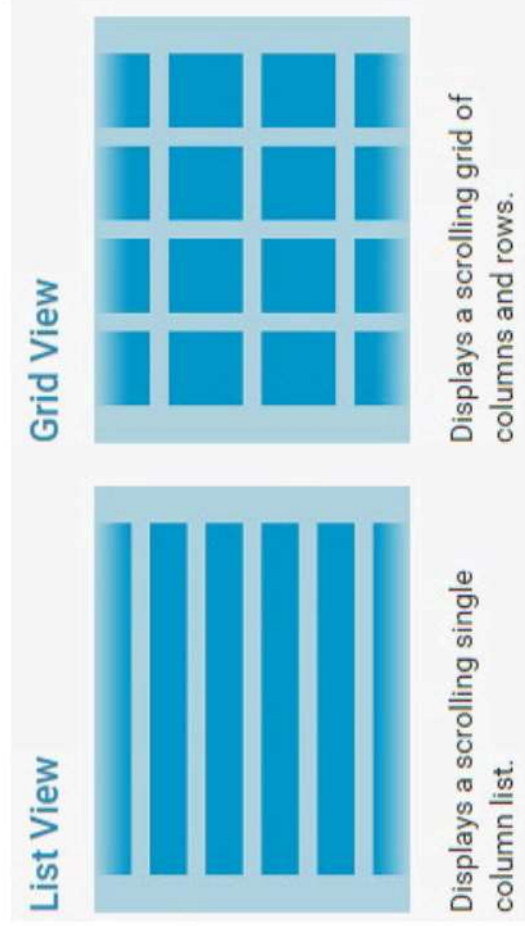
Layouts con Adapter

Cuando el contenido de un Layout no es fijo o no está predeterminado por el XML que lo soporta, se puede usar una subclase de **AdapterView**. El AdapterView usará un **Adapter**.

El **Adapter** funciona de intermediario entre el **AdapterView** y la fuente de datos.

El **Adapter** obtiene los datos y convierte cada uno (cada dato) en una **View** que puede ser añadida al **AdapterView**.

Algunos Layouts que se implementan con un **Adapter** son:



Rellenando el Adapter con Datos

Hay que vincular el **AdapterView** con el **Adapter**

Android proporciona subclases de **Adapter** para diferentes fuentes de datos, por ejemplo [ArrayAdapter](#) o [CursorAdapter](#)

Array Adapter

Cuando la fuente es un Array de datos.

Por defecto se crea una vista (un View) para cada elemento del array

llamando a **toString()** para cada uno y colocando el resultado en un **TextView**

```
ArrayAdapter<String> adapter = new ArrayAdapter<String>(
    this,
    android.R.layout.simple_list_item_1,
    myStringArray);
```

this: tu context. —————→ [¿Qué es el Context?](#)
el layout: que contiene un TextView para cada string
el array: de strings

En el ListView vinculamos el Adapter

```
ListView listView = (ListView) findViewById(R.id.listView);
listView.setAdapter(adapter);
```

Click events

Manejo de los Click Events en un AdapterView:

Puedes responder a los eventos de click en un **AdapterView** implementando un listener para `onItemClickListener`, es decir, implementando un **OnItemClickListener**:

```
// Create a message handling object as an anonymous class.  
private OnItemClickListener mMessageClickedHandler = new OnItemClickListener() {  
    public void onItemClick(AdapterView parent, View v, int position, long id) {  
        // Do something in response to the click  
    }  
};  
  
listView.setOnItemClickListener(mMessageClickedHandler);
```

umhandroid.momrach.es:

[ListView Simple – Respondiendo al `onClick` en el `ListView`](#)